Full-System FPGA Emulation

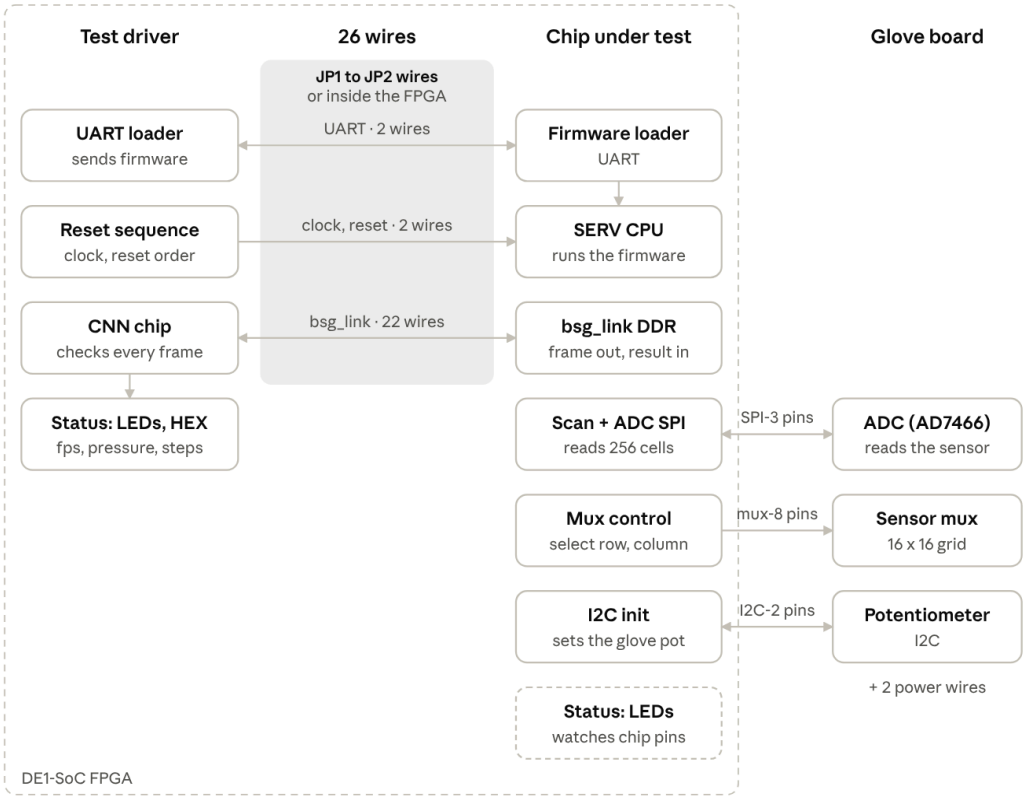
Team: Tactile Sensing Control
Name: Jeffery Wen
Date: November, 2026

1. Overview

This project continues the tactile-sensing chip work we simulated in EE478. I built a test driver for the chip and emulated the full system on one DE1-SoC FPGA at 50 MHz. The test driver and the chip RTL run side by side in one FPGA image, in two options: without loopback, connected inside the FPGA, and with loopback, connected through 26 jumper wires from header JP1 to JP2. The real glove and ADC board are also part of the test setup; the chip scans the real sensors through the mux, ADC, and I²C, so the test uses real pressure data, not a model.

2. Test Driver Design

- Block diagram



- I/O

Driver (JP1) ↔ chip (JP2) signals

Group	Signal	Header pin	Direction
Link into chip	dn_data[0..7]	1–8	driver → chip
Link into chip	dn_clk	9	driver → chip
Link into chip	dn_valid	10	driver → chip
Link credits	dn_token	13	chip → driver
Link credits	token_clk	14	driver → chip
Link out of chip	up_clk	15	chip → driver
Link out of chip	up_valid	16	chip → driver
Link out of chip	up_data[0..7]	17–24	chip → driver
Clock/reset	core_clk	25	driver → chip
Clock/reset	hard_reset	26	driver → chip
UART	uart_rx (firmware in)	27	driver → chip
UART	uart_tx (output out)	28	chip → driver

Chip ↔ glove board signals:

Group	Signal	FPGA pin	Glove board	Direction
Power	+3.3 V	JP2 29	3.3V	FPGA → glove
Power	GND	JP2 30	GND	—
Row mux	mux_row[0..3]	JP2 31–34	GND_SEL[0..3]	chip → glove
Column mux	mux_col[0..3]	JP2 35–38	RD_SEL[0..3]	chip → glove
I²C	i2c_scl	JP2 39	I2C_SCL	chip → glove
I²C	i2c_sda	JP2 40	I2C_SDA	both ways
ADC (SPI)	spi_sclk	JP1 31	ADC_SCLK	chip → glove
ADC (SPI)	spi_miso	JP1 32	ADC_MISO	glove → chip
ADC (SPI)	spi_cs_n	JP1 33	ADC_CS_n	chip → glove

Pins 11 (+5 V) and 12 (GND) are not used.

- Commands

Control	Down	Up
SW0	run	hold everything in reset
SW1	don’t send firmware (chip waits)	send firmware to the chip
SW2	without loopback	with loopback
SW9	LEDs show the driver	LEDs show the chip

LED	Step	Driver view (SW9 down)	Chip view (SW9 up)
0	Power	running	running
1	Clock	clock	clock
2	Firmware	firmware sent	firmware loaded
3	Boot	BOOT OK received	BOOT OK sent
4	Glove Setup	I²C write seen	I²C write seen
5	Sensor Data	Every frame had 256 pixels	ADC being read
6	Frame over link	Frame received	link sending
7	Result over link	result sent	link receiving
8	UART	UART receiving	UART sending
9	Result packet	Result packet received	Result packet sent

HEX	Shows
0-1	row/column of the strongest pressed point (0-F)
2-3	strongest pressure value in the last frame (00-FF)
4-5	Frames per second

Performance

Item	Value
Clock	50 MHz
UART	115200 baud
Frame rate	124 fps

Item	Value
Timing	Meets 50 MHz, worst slack +6.67 ns
FPGA use	9707 / 32070 ALMs (30%)

3. Tests

Test	What it checks	Simulation	Board
Firmware load	Chip replies ACK and BOOT OK	Pass	Pass
Without loopback	Full system inside the FPGA	Pass	Pass
With loopback	All 26 signals cross real pins and wires	Pass	Pass
Glove data	Every frame has all 256 pixels	Pass	Pass
Pressure	HEX shows the strongest point and follows a press	Pass	Pass
No firmware	chip waits and sends no frames	Pass	Pass
Speed	frames per second at 50 MHz	124 fps	124 fps

Debugging the pressure signal: Presses barely changed the reading at first. Longer settle times did not help, but re-powering the glove board did: at the pot's power-up gain (63), a fingertip press changes the reading by about 800 counts; at the chip's fixed boot value (125), only about 60. The right gain depends on the sensor and application, so it should be adjustable.

4. Results

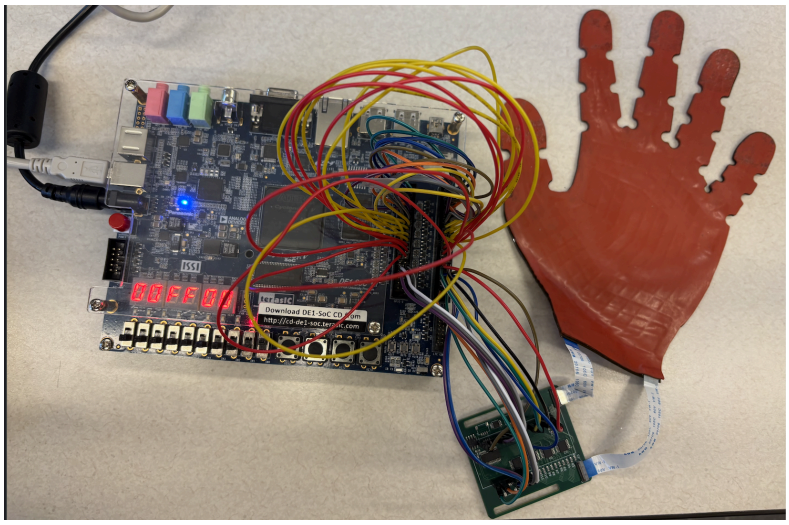


Figure 1. *In reset: only LED1 blinks to show the clock is running; the chip is held in reset*

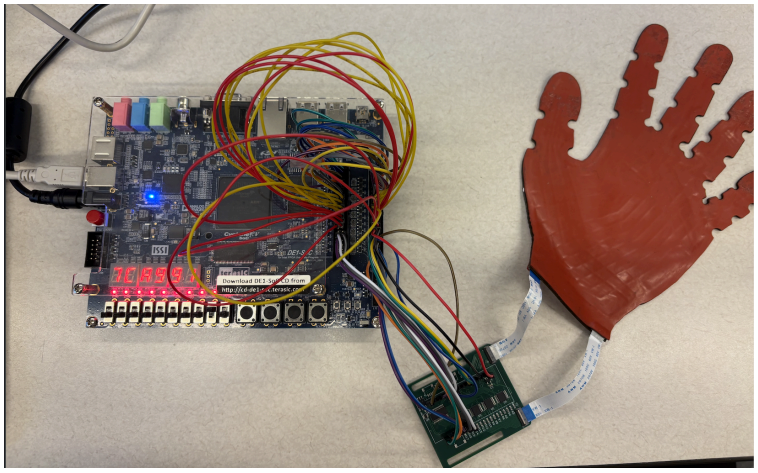


Figure 2. *Running without loopback: firmware loaded, BOOT OK, and frames flowing.*

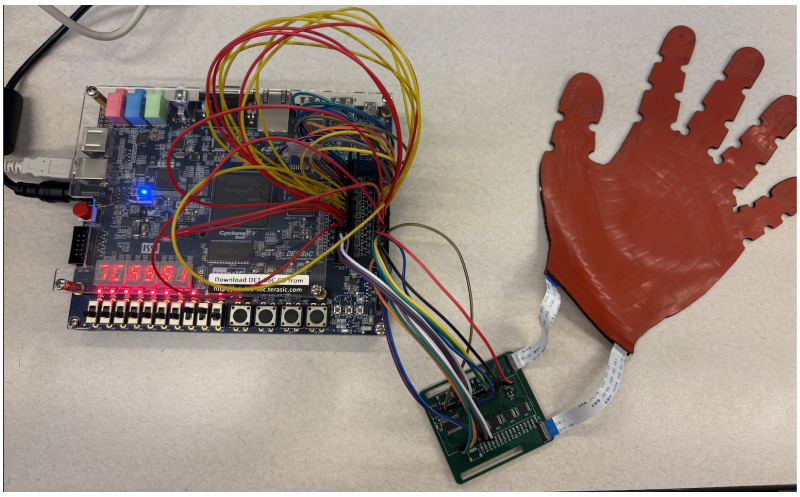


Figure 3. *Running with loopback: the same result with all 26 signals passing through the jumper wires*

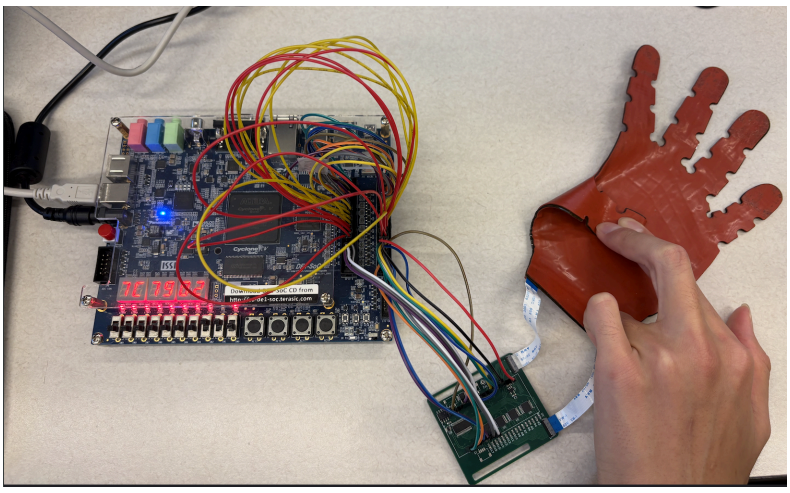


Figure 4. *Loopback with a press on the glove: HEX3–2 shows the pressure value drop*

5. Plan for real-chip testing

Real-chip testing follows the Assignment 1 plan, with this test driver wired to the chip board instead of the loopback wires: check for signs of life after power, clock and reset, test each block at a low clock, run the full system at low speed, then raise the clock toward 50 MHz to measure speed, frame rate and power. Since the chip's gain is fixed, the driver will rewrite the pot over I²C after boot to tune sensitivity for each sensor.

6. Lessons learned

- **Simulate the full board first:** Running the same design in simulation before wiring found firmware and reset problems early, so the first board test worked without hardware debugging.
- **Loopback helps debugging:** Being able to switch between the wires and the inside path shows right away whether a failure comes from the design or from the wiring.
- **The chip needs more visibility:** The scan chain can read out every flip-flop, but only after stopping the chip, and the UART only shows what the firmware prints. A live debug output, such as a status pin, would show where the chip gets stuck while it runs, although with only 48 pads, spare pins for debugging are limited.
- **Sensitivity should be adjustable:** The right gain depends on the sensor and the application. Our chip sets it once at boot to a fixed value, so it can't be tuned after tapeout; a firmware-writable gain would let the same chip work with different sensors.